

Modelos Generativos Profundos

Clase 9: Modelos autorregresivos y LLMs (parte V)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Propiedades agénticas

Eficiencia y compresión en inferencia

Fine tuning

Propiedades agénticas

Modelos conversacionales

Dada la capacidad de los modelos actuales y su capacidad de entender patrones y seguir instrucciones complejas, es posible construir un modelo conversacional usando **chat templates** (usualmente también se realiza fine tuning) de la forma

```
<|start|>
<|system|>
Eres un asistente útil y preciso que responde preguntas de cálculo.
Siempre entrega las derivadas paso a paso y utiliza notación matemática correcta.
<|end|>

<|user|>
¿Cuál es la derivada de  $x^3$ ?
<|end|>

<|assistant|>
La derivada de  $x^3$  es  $3x^2$ .
<|end|>

<|user|>
Gracias, y la derivada de  $x^x$ ?
<|end|>
```

También es posible acceder a información externa almacenada en bases de datos o en internet para complementar el conocimiento del LLM, disminuir las alucinaciones y mejorar sus respuestas.

Retrieval-Augmented Generation (RAG) consiste en codificar cada instancia de la base de datos en un embedding (p.g. usando un modelo tipo BERT) y luego guardar dicho embedding en una base de datos vectorial. De este modo:

- Dada una pregunta al LLM, se le calcula su embedding.
- Se buscan los $K \in \mathbb{N}$ embeddings más similares (p.g. usando similitud coseno) en la base de datos vectorial.
- Se agrega el texto asociado a estos embeddings al prompt que finalmente se entrega al LLM.

Uso de herramientas

Más en general, es posible que el LLM utilice **herramientas externas** para responder preguntas o realizar tareas.

Para esto, se le suele entregar al LLM un prompt (usando un formato bien definido) con instrucciones de la forma

Eres un asistente que puede utilizar las siguientes herramientas:

1. Calculadora: puedes usarla para realizar cálculos matemáticos.
2. Buscador web: puedes usarlo para buscar información en internet.

Cuando necesites usar una herramienta, responde con el formato:

```
<|tool|>Nombre de la herramienta: entrada<|end|>
```

Luego, espera a que te entreguen el resultado de la herramienta.

Cuando tengas toda la información necesaria, responde con el formato:

```
<|answer|>Tu respuesta aquí<|end|>
```

Un **parser** extrae la llamada a la herramienta y la respuesta del modelo.

Uso de herramientas

En general, el uso de herramientas requiere realizar fine tuning sobre el LLM para poder disminuir la probabilidad de que el modelo utilice mal las herramientas entregadas.

Por otro lado, se le suele pedir al modelo que el nombre de la herramienta a utilizar junto a sus parámetros sea entregado en formato JSON (para facilitar el parsing), el cual suele venir entre tokens especiales (p.g.<TOOL>), para facilitar su extracción.

También es posible pedirle al LLM que genere sus propias herramientas mediante código, las cuales luego puede ejecutar.

Modo agente

Dada la capacidad de planificación de los modelos, es posible construir un **agente** que planifique y ejecute acciones para cumplir un objetivo.

Una técnica estándar para esto es **Reasoning and Acting (ReAct)**, donde el agente genera una secuencia de pasos de la forma

Pensamiento: <...>.

Acción: <...>.

Observación: <...>.

Donde crea un plan de trabajo, ejecuta acciones (interrumpiendo temporalmente la generación), y luego añade al contexto del LLM los resultados obtenidos al ejecutar dichas acciones.

Este ciclo se repite hasta cumplir el objetivo.

Eficiencia y compresión en inferencia

Cantidad de parámetros y uso de memoria

Para una red neuronal cualquiera, se pueden medir dos aspectos de eficiencia con respecto a ella:

- **Cantidad de parámetros:** asociada a la cantidad de memoria que se necesita para almacenar el modelo.
- **Costo computacional de inferencia:** asociado a la cantidad de operaciones que se realizan al realizar un forward.

La cantidad de **VRAM** necesaria para realizar inferencia con un LLM escala (aproximadamente) de manera lineal con el número de parámetros del modelo.

Algunas técnicas de optimización permiten disminuir considerablemente la cantidad de VRAM necesaria.

Por otro lado, para el **entrenamiento** de un modelo es necesario contar con mucha más memoria y poder computacional.

Cantidad de parámetros y uso de memoria

Contando la cantidad de parámetros de la arquitectura GPT se observa que:

- El bloque FF tiene más parámetros que el bloque MHSA.
- Si $|\mathcal{V}|$ es muy grande, la cantidad de parámetros en la capa de embedding y en el cabezal de lenguaje final se puede volver significativa.
- Si cada parámetro se almacena con precisión **Float 32**, la cantidad de VRAM que se necesita es $\frac{\text{nº parámetros} \times 4}{1024 \times 1024}$ GB.
- El cálculo de la atención tiene un **costo cuadrático** en el largo de la secuencia.

KV caching

Dada una secuencia de embeddings $S_{\text{in}} = (x_1, \dots, x_T) \in \mathbb{R}^D$, la arquitectura GPT (previo al cabezal de lenguaje) genera otra secuencia $S_{\text{out}} = (y_1, \dots, y_T) \in \mathbb{R}^D$.

Durante la generación con un LLM, solo es necesario calcular $y_T \in \mathbb{R}^D$ ya que $p_\theta(w_{T+1}|w_1, \dots, w_T) \sim \text{Categórica}(\mathcal{V}; r_\theta)$, donde $r_\theta \propto W_{\text{out}} y_T$. El vector $y_T \in \mathbb{R}^D$ se calcula como:

$$y_T = \sum_{t=1}^T \frac{\exp\left(\frac{\langle q_T, k_t \rangle}{\sqrt{P}}\right)}{\sum_{r=1}^T \exp\left(\frac{\langle q_T, k_r \rangle}{\sqrt{P}}\right)} v_t$$

Por lo que los vectores $\{k_1, \dots, k_{T+1}\}, \{v_1, \dots, v_{T+1}\} \subset \mathbb{R}^P$ pueden ir siendo almacenados en memoria para no tener que volver a calcularlos. Esta técnica se conoce como **KV caching**.

Cuantización

- En una regresión lineal unidimensional, $y(x) = ax + b$, es posible modificar levemente la precisión de los parámetros $a, b \in \mathbb{R}$ sin afectar significativamente la predicción $y(x) \in \mathbb{R}$.
- Esta intuición se puede extender para reducir la precisión de los parámetros de un LLM sin afectar significativamente su rendimiento. Esta técnica se conoce como **cuantización**.
- La cuantización permite reducir la cantidad de memoria necesaria para almacenar un modelo, a costa de una leve pérdida en su rendimiento.

Destilación

Es posible entrenar un modelo más pequeño (**student**) para que imite el comportamiento de un modelo más grande (**teacher**). Esta técnica se conoce como **destilación**.

La destilación se puede realizar de varias maneras, por ejemplo:

- **Soft targets:** en lugar de entrenar al modelo student con las hard-targets (one-hot) del trainset, se utilizan las distribuciones de probabilidad (soft targets) generadas por el teacher.
- **Feature distillation:** se extraen características intermedias del teacher y se utilizan para guiar el entrenamiento del student.

También es posible destilar modelos tipo MoE entrenando un GPT estándar para que imite el comportamiento del modelo MoE. Esta técnica se conoce como **densification**.

Pruning

Se ha demostrado empíricamente que algunas capas de un Transformer no modifican de manera considerable los embedding de la secuencia de entrada, por lo que influyen menos en el comportamiento del modelo al momento de hacer inferencia.

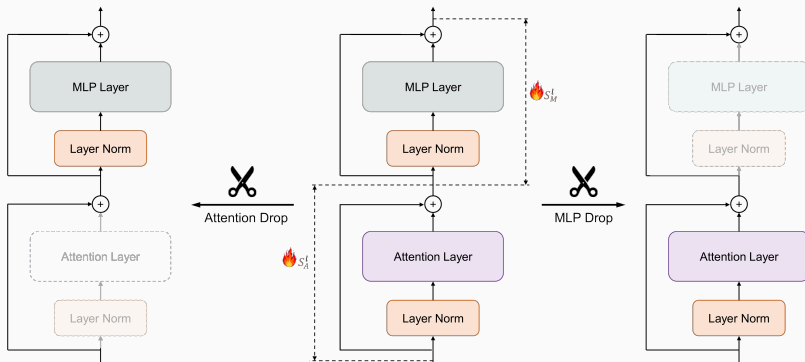
Más aún, es posible remover algunas capas (**pruning**) de un modelo tipo Transformer ya entrenado sin afectar significativamente su rendimiento.

Si $\text{score}(S_{\text{in}}, S_{\text{out}}) = 1 - \text{SimilitudCoseno}(S_{\text{in}}, S_{\text{out}})$ mide la *importancia* de una capa, entonces un criterio natural para realizar pruning es remover las capas con menor score.

Por ejemplo, se puede remover la mitad de las capas de atención en LLaMA 2 y perder solo un 2.4% de rendimiento mientras que gana casi un 50% de aceleración en inferencia.

Pruning

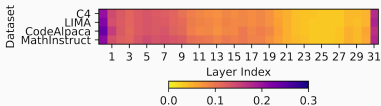
Se tienen 3 opciones naturales de pruning en un bloque Transformer:



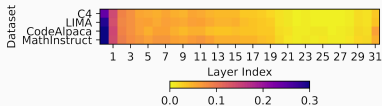
Pruning

Ordenando todas las capas de un modelo Transformer por su score de importancia, se observa que:

- Muchas capas de atención tienen poca importancia. Más aún, eliminarlas no afecta significativamente el rendimiento.
- También es posible eliminar algunas capas FF junto a las capas de atención para mejorar aún más la eficiencia.
- Este patrón se observa a lo largo de todo el entrenamiento.



(a) MLP



(b) Attention

Fine tuning

Fine tuning

Si bien la técnica de prompting es muy potente, algunas tareas aún requieren ajustar un poco más el modelo para lograr comportamientos más complejos o menos aleatorios.

Una forma de lograr esto es haciendo **fine tuning**, lo cual consiste en continuar el pre-entrenamiento del modelo utilizando un dataset específico para la tarea en la que se desea mejorar.

El fine tuning puede provocar una pérdida de rendimiento en otras tareas e incluso puede provocar que el LLM olvide información aprendida anteriormente (**catastrophic forgetting**).

Esto último puede ser aminorado agregando al fine tuning un poco de data usada durante el pre-entrenamiento.

Instruction fine-tuning (SFT)

Se puede hacer fine tuning utilizando datasets de instrucciones y respuestas siguiendo un **prompt template** de la forma

“Instrucción: <...>. Respuesta: <...>.”

Una vez entrenado, este modelo se utiliza entregándole como input un prompt de la forma “Instrucción: <...>. Respuesta: ”.

También es posible obtener modelos conversacionales haciendo fine tuning con datasets de diálogos formateados con un chat template predefinido.

El chat template usado puede incluir estructuras adicionales como system prompts, contextos y uso de herramientas.

Para hacer fine tuning, usualmente solo se actualizan los parámetros del cabezal de lenguaje y de las últimas capas del modelo.

Sin embargo, es posible actualizar todos los parámetros de forma eficiente utilizando técnicas como **Low-Rank Adaptation (LoRA)**.

Esta técnica permite hacer un fine tuning menos costoso sobre capas neuronales lineales (i.e., de la forma $x \mapsto Wx$), como las que se observan en los bloques MHSA y FF de un Transformer.

Más precisamente, LoRA reparametriza la actualización ΔW de la matriz de parámetros fine-tuneada, $\tilde{W} = W + \Delta W$, como el producto de dos matrices de menor tamaño. De este modo, solo se deben entrenar estas dos matrices pequeñas en lugar de la matriz completa ΔW .

Si $x \in \mathbb{R}^n \mapsto (W + \Delta W)x \in \mathbb{R}^m$ es la capa lineal fine-tuneada, con $W \in \mathcal{M}_{mn}(\mathbb{R})$ la matriz de parámetros original y $\Delta W \in \mathcal{M}_{mn}(\mathbb{R})$ el *shift* que se aprende durante el fine tuning, LoRA propone reformular este shift como

$$\Delta W = AB,$$

donde $A \in \mathcal{M}_{m \times r}(\mathbb{R})$, $B \in \mathcal{M}_{r \times n}(\mathbb{R})$ son matrices entrenables. De este modo, ahora solo se tienen que ajustar $mr + rn$ parámetros en lugar de mn , por lo que si $r \ll \frac{mn}{m+n}$, la cantidad de parámetros a ajustar es mucho menor.

En consecuencia, LoRA sustituye la capa lineal original por un módulo de la forma

$$x \mapsto Wx + ABx,$$

donde la matriz W está fija y solo se entrena A y B .

En la próxima clase.

- **Introducción a las GANs:** formulación, implementación, generación condicional.

Modelos Generativos Profundos

Clase 9: Modelos autorregresivos y LLMs (parte V)